

Module-04**RBT Level-L1, L2****Object Detection and Recognition:**

Introduction to object detection and recognition, Introduction to HAAR cascades, YOLO, SSD, SIFT, ORB and FPN Transfer learning for object detection, motion detection, 3D vision: Depth estimation, stereo vision, Structure from Motion.

Introduction to object detection and recognition

Object detection primarily aims to answer two critical questions about any image: "Which objects are present?" and "Where are these objects situated?" This process involves both object classification and localization:

Classification: This step determines the category or type of one or more objects within the image, such as a dog, car, or tree.

Localization: This involves accurately identifying and marking the position of an object in the image, typically using a bounding box to outline its location.

Key Components of Object Detection**1. Image Classification**

Image classification assigns a label to an entire image based on its content. While it's a crucial step in understanding visual data, it doesn't provide information about the object's location within the image.

2. Object Localization

Object localization goes a step further by not only identifying the object but also determining its position within the image. This involves drawing bounding boxes around the objects.

3. Object Detection

Object detection merges image classification and localization. It detects multiple objects in an image, assigns labels to them, and provides their locations through bounding boxes.

How Object Detection works?

The general working of object detection is:

Input Image: the object detection process begins with image or video analysis.

Pre-processing: image is pre-processed to ensure suitable format for the model being used.

Feature Extraction: CNN model is used as feature extractor, the model is responsible for dissecting the image into regions and pulling out features from each region to detect patterns of different objects.

Classification: Each image region is classified into categories based on the extracted features. The classification task is performed using SVM or other neural network that computes the probability of each category present in the region.

Localization: Simultaneously with the classification process, the model determines the bounding boxes for each detected object. This involves calculating the coordinates for a box that encloses each object, thereby accurately locating it within the image.

Non-max Suppression: When the model identifies several bounding boxes for the same object, non-max suppression is used to handle these overlaps. This technique keeps only the bounding box with the highest confidence score and removes any other overlapping boxes.

Output: The process ends with the original image being marked with bounding boxes and labels that illustrate the detected objects and their corresponding categories.

Techniques in Object Detection

Traditionally, the task of object detection relied on manual feature extraction and classification. Some of the traditional methods are:

- Haar Cascades
- Histogram of Oriented Gradients (HOG)
- SIFT (Scale-Invariant Feature Transform)

Deep Learning Methods for Object Detection

Deep learning played an important role in revolutionizing the computer vision field. There two primary types of object detection methods:

Two-Stage Detectors: These detectors work in two stages: first, they will propose candidate region and then classify the region into categories. Some of the two stage detectors are R-CNN, Fast R-CNN and Faster R-CNN.

Single-stage Detectors: In a single pass, these detectors accurately forecast the bounding boxes and class probabilities for every area of the picture. YOLO (You Only Look Once) and SSD (Single Shot MultiBox Detector) are two examples.

Two-Stage Detectors for Object Detection

There are three popular two-stage object detection techniques:

1. R-CNN (Regions with Convolutional Neural Networks)

This technique uses selective search algorithm to generate 2000 region proposals from an image, then the proposed region is resized and passed through pre-trained CNN based models to extract feature vectors. Then, these feature vectors are fed to the classifier for classifying object within the region.

2. Fast R-CNN

This technique processes the complete image with the CNN to produce a feature map. Region of Interest Pooling layers is used to extract the feature vector from the feature map. The techniques utilizes integrated classification and regression approach, it use uses a single fully connected network to provide the output for both the class probabilities and bounding box coordinates.

3. Faster R-CNN

This technique utilizes Region Proposal Network (RPN) that predicts the object bounds from the feature maps created by the initial CNN then, the features of the proposed region generated by RPM are pooled using ROI Pooling and fed into a network that predict the class and bounding box.

Single-Stage Detectors for Object Detection

Single-stage detectors focuses on merging the object localization and classification tasks into single pass through neural network. There are two popular models for single-stage object detection:

1. SSD (Single Shot MultiBox Detector)

Using feature maps at various sizes, SSD (Single Shot MultiBox Detector) is a one-stage object detection architecture that predicts item bounding boxes and class probabilities immediately. It is quicker and more effective than two-stage methods as it makes use of a single deep neural network to do both object identification and area proposal at the same time.

2. YOLO (You Only Look Once)

YOLO, or "You Only Look Once," is an additional one-stage object identification architecture that uses whole photos to forecast class probabilities and

bounding boxes in a single run. It provides very accurate object recognition in real time by dividing the input picture into a grid and predicting bounding boxes and class probabilities for each grid cell. The process is discussed below:

Detection in a single step: YOLO formulates the issue of object detection as a regression and uses a single network assessment to forecast both class probabilities and bounding box coordinates.

Grid-based Detection: An input picture is split into grid cells, and for each item included in a grid cell, bounding boxes and class probabilities are predicted.

Applications of Object Detection

1. Autonomous Vehicles

Object detection is crucial for the safe operation of autonomous vehicles, allowing them to perceive their surroundings, detect pedestrians, other vehicles, and obstacles, and make real-time decisions to ensure safe navigation.

2. Security and Surveillance

Object detection enhances security systems by enabling the identification of suspicious activities, intruders, and overall surveillance efficiency.

3. Healthcare

Object detection assists in medical imaging, helping to detect abnormalities such as tumors in X-rays and MRIs, thus contributing to accurate and timely diagnoses.

4. Retail

In retail, object detection automates inventory management, prevents theft, and analyzes customer behavior, enhancing operational efficiency and customer experience.

5. Robotics

Object detection enables robots to interact with their environment, recognize objects, and perform tasks autonomously, significantly enhancing their functionality.

Introduction to HAAR cascades.

Haar Cascade classifiers are a machine learning-based method for object detection. They use a set of positive and negative images to train a classifier, which is then used to detect objects in new images.

- **Positive Images:** These images contain the objects that the classifier is trained to detect.
- **Negative Images:** These images contain everything else which do not contain the object we want to detect.

Code Implementation

```
import cv2
```

```
# Step 1: Load the Haar Cascade Classifier
```

```
face_cascade = cv2.CascadeClassifier(cv2.data.harcascades +  
'haarcascade_frontalface_default.xml')
```

```
# Step 2: Read the image
```

```
img = cv2.imread('group_photo.jpg') # Replace with your image path
```

```
# Step 3: Convert image to grayscale
```

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

```
# Step 4: Detect faces
```

```
faces = face_cascade.detectMultiScale(  
    gray,  
    scaleFactor=1.1,  
    minNeighbors=5,  
    minSize=(30, 30)  
)
```

```
print(f' Found {len(faces)} face(s).')
```

```
# Step 5: Draw rectangles around detected faces
```

```
for (x, y, w, h) in faces:
```

```
    cv2.rectangle(img, (x, y), (x + w, y + h), (0, 255, 0), 2)
```

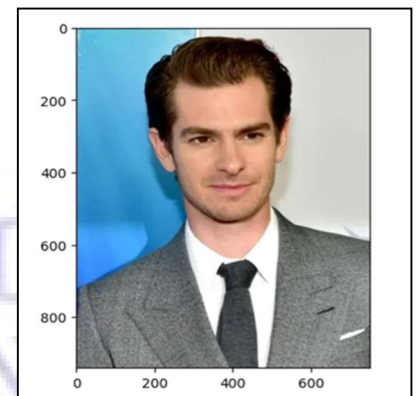
```
# Step 6: Display the result
```

```
cv2.imshow('Detected Faces', img)
```

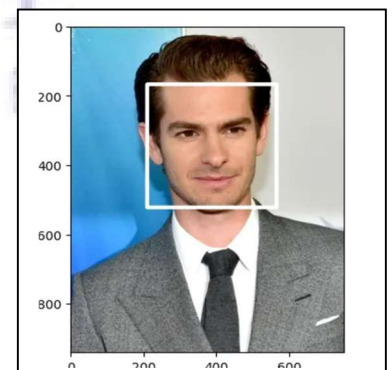
```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

Input image



Output image(Face detected)



YOLO (You Only Look Once)

The core of YOLO's architecture is a single convolutional neural network (CNN) that simultaneously predicts multiple bounding boxes and class probabilities for each bounding box. The key components of YOLO's architecture are:

- **Grid Division:** The input image is divided into an $S \times S$ grid. Each grid cell is responsible for detecting objects whose center falls within it.
- **Bounding Box Prediction:** Each grid cell predicts B bounding boxes, each with a confidence score. The confidence score reflects the probability that a bounding box contains an object and the accuracy of the bounding box.
- **Class Prediction:** Each grid cell also predicts the class probabilities C for the object, given that an object exists in that grid cell.
- **Final Predictions:** YOLO combines these predictions to generate the final detected objects, applying non-max suppression to eliminate redundant bounding boxes.

How YOLO Works for Object detection?

- **Image Input:** The input image is resized to a fixed size (e.g., 448 x 448) to ensure consistent processing.
- **Grid Division:** The image is divided into a grid (e.g., 7 x 7). Each cell in the grid is responsible for predicting objects that fall within it.
- **Feature Extraction:** The CNN extracts features from the image using convolutional layers. These features are used to predict bounding boxes and class probabilities.
- **Bounding Box Prediction:** Each grid cell predicts multiple bounding boxes. Each bounding box has five components: x, y (coordinates of the center), w, h (width and height), and the confidence score.
- **Class Prediction:** Each grid cell predicts the probabilities for each class (e.g., person, car, dog) of the object present.
- **Combining Predictions:** YOLO combines the bounding box predictions and class probabilities to generate a final list of detected objects.
- **Non-Max Suppression:** To handle multiple detections of the same object, YOLO applies non-max suppression, which removes redundant bounding boxes based on their confidence scores.

Code Implementation of YOLO for Object detection

Step 1: Install Required Libraries

```
pip install torch torchvision torchaudio
pip install opencv-python
pip install numpy
pip install matplotlib
pip install pyyaml
pip install tqdm
```

Step 2: Clone the YOLOv5 Repository

```
git clone https://github.com/ultralytics/yolov5
cd yolov5
pip install -r requirements.txt
```

Step 3: Import Required Libraries

```
import torch
from pathlib import Path
import cv2
import matplotlib.pyplot as plt
import numpy as np
```

Step 4: Load the Pre-trained YOLOv5 Model

```
model = torch.hub.load('ultralytics/yolov5', 'yolov5s',
pretrained=True)
```

Step 5: Load and Preprocess the Input Image

```
# Load image
img_path = 'path/to/your/image.jpg'
img = cv2.imread(img_path)

# Convert BGR to RGB
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Display the image
plt.imshow(img)
plt.show()
```

Step 6: Perform Object Detection

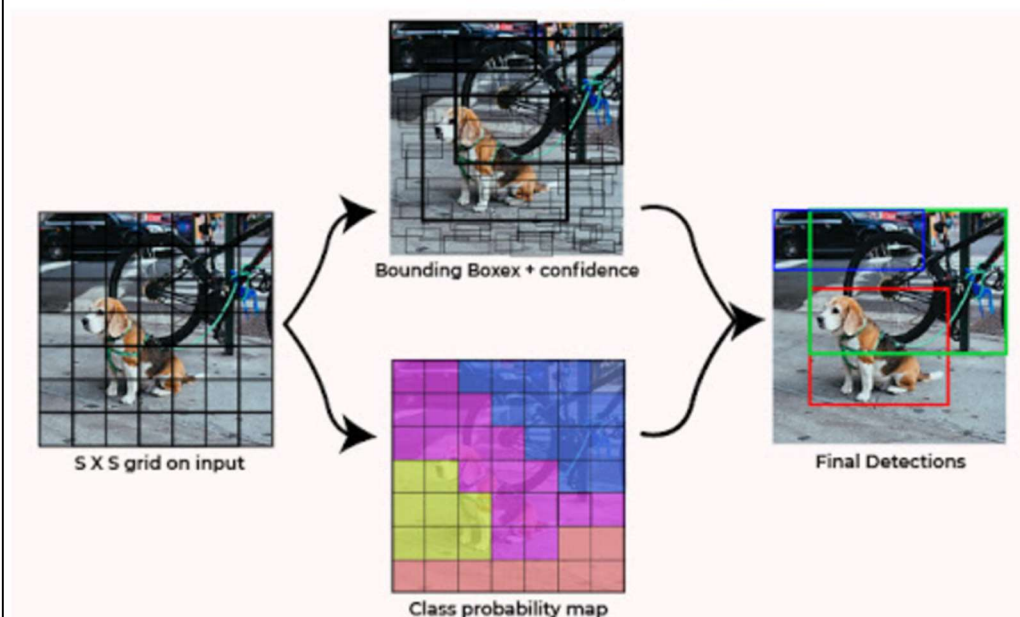
```
# Perform inference
results = model(img)

# Print results
results.print() # Print results to console
results.show() # Display results
```

Step 7: Display Detection Results

```
# Get the results
detections = results.pandas().xyxy[0] # Results as pandas dataframe

# Print detections
print(detections)
```

Output:**Oriented FAST and Rotated BRIEF(ORB)**

The ORB (Oriented FAST and Rotated BRIEF) algorithm is an efficient method for feature matching. It combines FAST which detects keypoints and BRIEF which describes those keypoints. The ORB (Oriented FAST and Rotated BRIEF) algorithm is an efficient method for feature matching. It combines FAST which detects keypoints and BRIEF which describes those keypoints.

Key Features of ORB

Fast and Efficient: ORB is optimized for speed, making it suitable for real-time applications. Its fast processing ensures quick feature detection and matching without compromising performance.

Rotation Invariant: One of the major challenges with feature matching is dealing with image rotation. It addresses this by making the BRIEF descriptors rotation-invariant. This means it can successfully match features even if the images are rotated.

Multiscale Detection: It can detect features at different scales using an image pyramid. This allows the algorithm to handle varying image sizes, making it robust in situations where the images are zoomed in or out.

To Perform Feature Matching using ORB in Python.

Step 1: Installing required libraries

Make sure we have OpenCV installed

Step 2: Importing Libraries

```
import numpy as np
import cv2
import matplotlib.pyplot as plt
from google.colab.patches import cv2_imshow
```

Step 3: Loading the Images

```
query_img = cv2.imread('/content/sports-car test.webp')
train_img = cv2.imread('/content/sports-car train.webp')

query_img_bw = cv2.cvtColor(query_img, cv2.COLOR_BGR2GRAY)
train_img_bw = cv2.cvtColor(train_img, cv2.COLOR_BGR2GRAY)
```

Step 4: Detecting Keypoints and Finding Descriptors

```
orb = cv2.ORB_create()

queryKeypoints, queryDescriptors =
orb.detectAndCompute(query_img_bw, None)
trainKeypoints, trainDescriptors =
orb.detectAndCompute(train_img_bw, None)
```

Step 5: Matching the Descriptors

```
final_img = cv2.drawMatches(query_img, queryKeypoints,  
                             train_img, trainKeypoints,  
                             matches[:20], None)  
final_img = cv2.resize(final_img, (1000, 650))
```

Step 6: Visualizing the Matches

```
final_img = cv2.drawMatches(query_img, queryKeypoints,  
                             train_img, trainKeypoints,  
                             matches[:20], None)  
final_img = cv2.resize(final_img, (1000, 650))  
  
plt.figure(figsize=(10, 6))  
plt.imshow(cv2.cvtColor(final_img, cv2.COLOR_BGR2RGB))  
plt.title("Feature Matches")  
plt.axis('off')  
plt.show()
```

Output

Feature Matches



Applications of ORB

- **Object Recognition:** Identifying objects in different images, even if they are rotated or scaled differently.
- **Image Stitching:** Combining multiple images to form a panorama by matching common features.
- **Augmented Reality:** Matching real-world scenes to virtual objects by identifying key features in camera feeds.
- **3D Reconstruction:** Matching features from different views to construct 3D models.

Limitations of ORB

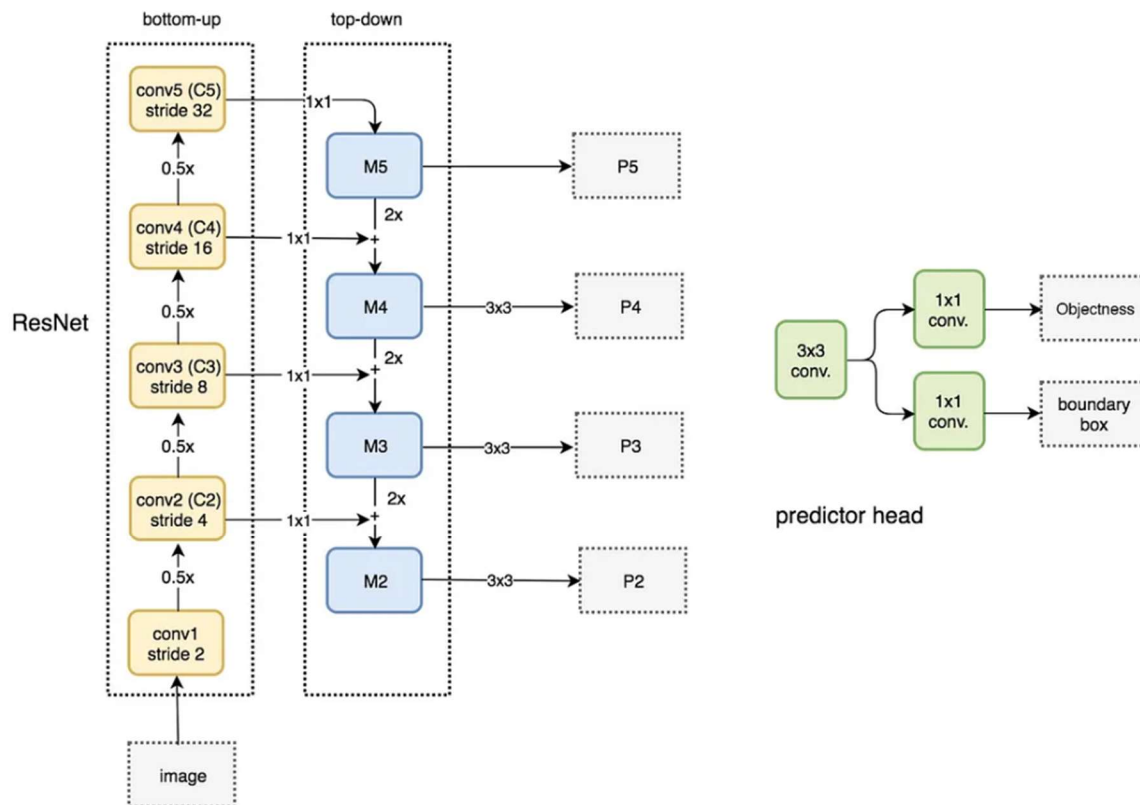
- **Less Accurate in Low-texture Areas:** ORB may not perform as well in areas with fewer distinct features or low texture.
- **Sensitivity to Noise:** Like many feature detectors, it can be sensitive to noise in the images.
- **No Descriptor Matching Optimization:** While it is efficient, the matching process could still be optimized further, especially for large datasets.

FPN Transfer learning for object detection

FPN (Feature Pyramid Network) transfer learning involves using a pre-trained FPN model on a new, custom dataset.

The FPN architecture for transfer learning

The FPN model architecture has two main pathways that create a multi-scale feature pyramid, which is crucial for detecting objects of various sizes.



Bottom-up pathway: This is the standard convolutional backbone (like a ResNet or EfficientNet) that processes the input image. As the network gets deeper, the spatial resolution of the feature maps decreases, but the semantic information becomes richer.

Top-down pathway: This pathway upsamples the semantically rich but low-resolution feature maps from the deeper layers.

Lateral connections: These connections merge the upsampled feature maps with the corresponding higher-resolution feature maps from the bottom-up pathway. This fusion helps combine high-level semantic information with low-level spatial details, which improves the detection of small objects.

Workflow for FPN transfer learning in practice

- **Prepare your dataset:** Collect and label your image data, ensuring the annotations are in the correct format (e.g., COCO or Pascal VOC).
- **Choose a pre-trained model:** Select a detection model that uses an FPN architecture, such as Faster R-CNN, Mask R-CNN, or RetinaNet, with a standard backbone like ResNet.

- **Configure the training environment:** Use a deep learning framework like TensorFlow, PyTorch, or a library like MMDetection, which simplifies the process of using and configuring FPN models.
- **Implement your transfer learning strategy:**
 - ❖ **For feature extraction:** Load the model, freeze the backbone and FPN layers, and train only the newly added detection head.
 - ❖ **For fine-tuning:** Load the model, unfreeze the final FPN and backbone stages, and train with a very low learning rate.
- **Train and evaluate:** Train the model on your custom dataset and evaluate its performance using metrics like mean Average Precision (mAP).
- **Iterate and optimize:** If the performance is not satisfactory, try different strategies, such as unfreezing more layers or adjusting hyperparameters like the learning rate.

Motion Detector in Python

```
# Python program to implement
# Webcam Motion Detector
# importing OpenCV, time and Pandas library
import cv2, time, pandas

# importing datetime class from datetime library
from datetime import datetime

# Assigning our static_back to None
static_back = None
# List when any moving object appear
motion_list = [ None, None ]
# Time of movement
time = []
# Initializing DataFrame, one column is start
# time and other column is end time
df = pandas.DataFrame(columns = ["Start", "End"])
# Capturing video
video = cv2.VideoCapture(0)
# Infinite while loop to treat stack of image as video
while True:
# Reading frame(image) from video
check, frame = video.read()
# Initializing motion = 0(no motion)
motion = 0
# Converting color image to gray_scale image
```

```
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
# Converting gray scale image to GaussianBlur
# so that change can be find easily
gray = cv2.GaussianBlur(gray, (21, 21), 0)
# In first iteration we assign the value
# of static_back to our first frame
if static_back is None:
    static_back = gray
    continue
# Difference between static background
# and current frame(which is GaussianBlur)
diff_frame = cv2.absdiff(static_back, gray)
# If change in between static background and
# current frame is greater than 30 it will show white color(255)
thresh_frame = cv2.threshold(diff_frame, 30, 255, cv2.THRESH_BINARY)[1]
thresh_frame = cv2.dilate(thresh_frame, None, iterations = 2)
# Finding contour of moving object
cnts,_ = cv2.findContours(thresh_frame.copy(),
                          cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
for contour in cnts:
    if cv2.contourArea(contour) < 10000:
        continue
    motion = 1
    (x, y, w, h) = cv2.boundingRect(contour)
# making green rectangle around the moving object
cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 3)
# Appending status of motion
motion_list.append(motion)
motion_list = motion_list[-2:]
# Appending Start time of motion
if motion_list[-1] == 1 and motion_list[-2] == 0:
    time.append(datetime.now())
# Appending End time of motion
if motion_list[-1] == 0 and motion_list[-2] == 1:
    time.append(datetime.now())
# Displaying image in gray_scale
cv2.imshow("Gray Frame", gray)
# Displaying the difference in currentframe to
# the staticframe(very first_frame)
cv2.imshow("Difference Frame", diff_frame)
# Displaying the black and white image in which if
# intensity difference greater than 30 it will appear white
cv2.imshow("Threshold Frame", thresh_frame)
# Displaying color frame with contour of motion of object
cv2.imshow("Color Frame", frame)
key = cv2.waitKey(1)
```

```

# if q entered whole process will stop
if key == ord('q'):
# if something is moving then it append the end time of movement
    if motion == 1:
        time.append(datetime.now())
    break
# Appending time of motion in DataFrame
for i in range(0, len(time), 2):
    df = df.append({"Start":time[i], "End":time[i + 1]}, ignore_index = True)
# Creating a CSV file in which time of movements will be saved
df.to_csv("Time_of_movements.csv")
video.release()
# Destroying all the windows
cv2.destroyAllWindows()

```

3D vision: Depth estimation, stereo vision, Structure from Motion.

3D vision uses techniques like stereo vision (two cameras) and Structure from Motion (moving camera) to estimate depth and reconstruct a 3D scene from 2D images.

Stereo vision

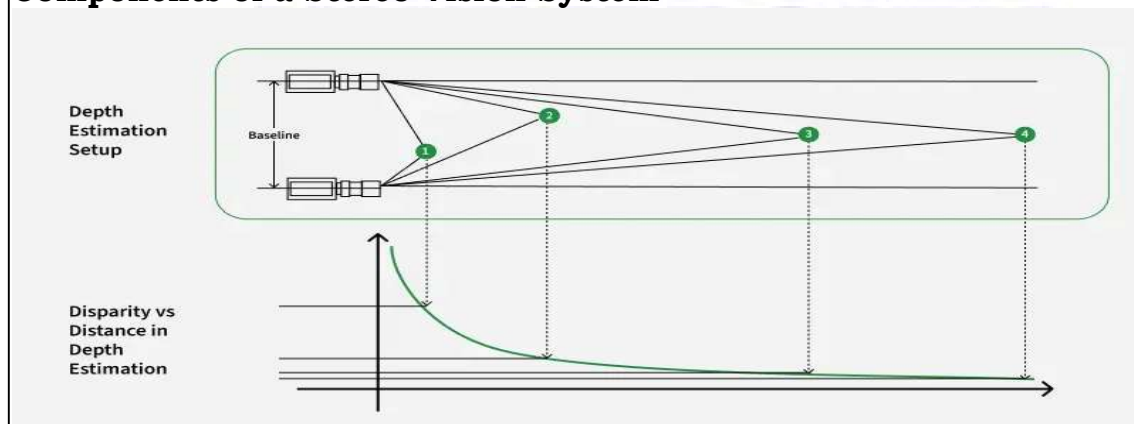
Concept: Uses two or more cameras from different viewpoints to mimic human binocular vision.

Process:

- Two cameras capture the same scene from slightly offset positions. Algorithms find corresponding points in both images.
- The "disparity" (difference in position) between these points is used to calculate depth, as a larger disparity means the object is closer.
- This can be summarized with the formula:

$$(\text{Depth}(Z) = \{(\text{focal_length} * \text{baseline}) / \{\text{disparity}\}\})$$

Components of a Stereo Vision System



Stereo Camera Setup: Two cameras with a known and fixed baseline.

Camera Calibration: Intrinsic and extrinsic parameter estimation for accurate geometry.

Image Rectification: Aligns image planes so epipolar lines are horizontal.

Feature Detection / Stereo Matching: Detects or matches corresponding points (using methods like SGM or block matching).

Disparity Estimation: Measures pixel shifts (disparity) between the two images.

Depth Computation: Calculates depth using triangulation.

Depth Map Generation: Produces a pixel-wise 2D array representing distances.

3D Reconstruction (Optional): Converts disparity and camera parameters into 3D coordinates or point clouds.

Advantages of Stereovision

- No need for active sensors.
- Relatively low-cost with commercial stereo cameras.
- Useful in both indoor and outdoor environments.
- Good depth resolution for nearby objects.
- Can be used in real-time systems.

Applications of Stereovision

- Autonomous Vehicles for obstacle detection and navigation.
- Robotics for grasping and interaction with environments.
- Medical Imaging.
- Industrial Inspection.

S.No	Questions
01	List & Explain the Key Components of Object Detection
02	Explain the general working of object detection
03	Explain techniques involved in Object Detection
04	Explain Two-Stage Detectors for Object Detection
05	Explain Single-Stage Detectors for Object Detection
06	List & explain the applications of Object Detection
07	Write python code for HAAR cascades
08	Explain the working principle of YOLO for Object detection
09	Write python code for YOLO for Object detection
10	Define Oriented FAST and Rotated BRIEF(ORB) & mention its key features.
11	Write python code for ORB
12	Mention the Applications & Limitations of ORB
13	With neat block diagram Explain FPN Transfer learning for object detection
14	Write python code for Motion Detector
15	With neat block diagram Explain Stereo Vision System